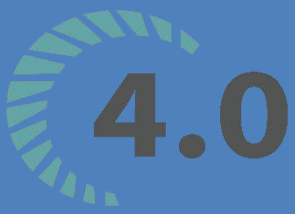
BỘ MÔN HỆ THỐNG THÔNG TIN – KHOA CÔNG NGHỆ THÔNG TIN

ĐẠI HỌC KHOA HỌC TỰ NHIÊN THÀNH PHỐ HỒ CHÍ MINH, ĐẠI HỌC QUỐC GIA TP HCM

MÔN HỌC
HỆ QUẢN TRỊ CƠ SỞ DỮ LIỆU



GVHD: Ths. TIẾT GIA HỒNG

BÁO CÁO ĐỒ ÁN MÔN HỌC - HỆ QUẢN TRỊ CƠ SỞ DỮ LIỆU

HỌC KỲ I – NĂM HỌC 2025-2026

KHOA CÔNG NGHỆ THÔNG TIN
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
227 Nguyễn Văn Cừ, Phường 4, Quận 5, TP.HCM
Điện Thoại: (08) 38.354.266 - Fax:(08) 38.350.096

THÔNG TIN NHÓM

STT	MSSV	Họ tên	Công việc	% Hoàn thành
1	23120028	Nguyễn Hoàng Danh	- Xử lí tình huống tranh chấp (Deadlock)	100
2	23120087	Phạm Ngọc Thanh	- Xử lí tình huống tranh chấp (Lost update)	100
3	23120361	Đào Đức Thịnh	- Xử lí Cài đặt tình huống tranh chấp (Dirty read)	100
4	23120405	Đỗ Phước Vinh (Nhóm trưởng)	- Xử lí tình huống tranh chấp (Phantom) - Báo cáo	100
5	23120411	Thạch Ý	- Xử lí tình huống tranh chấp (Unrepeatable read)	100

XỬ LÍ TÌNH HUỐNG TRANH CHẤP

I. Sinh viên thực hiện: 23120405 - Đỗ Phước Vinh

1. **Tình huống:** Khách hàng A đặt sân B online (Thêm đơn đặt sân). Lễ tân kiểm tra sân đã đặt hay chưa.

ERR01: ***Phantom Read***

T1 (User = Khách hàng online): Thực hiện đặt sân B

T2 (User = Nhân viên lễ tân): Thực hiện kiểm tra sân trống, xem danh sách sân đã đặt

sp_DatSan	**Khóa**	**sp_KiemTraSanDat**	**Khóa**
Input*:** @CourtID @OrderDate @OrderTime ***Output: (1 - Đặt thành công, 0 - Đặt thất bại)		***Input***: @CourtID @StartTime @EndTime ***Output***:: Các bản ghi trong **BOOKING_FORM** thỏa điều kiện input	
		SET TRANSACTION ISOLATION LEVEL SERIALIZABLE	
		BEGIN TRAN	
	.	DECLARE @d DATE = CONVERT(date, @StartTime); **Xem danh sách các sân đã đặt** SELECT BookingID, CourtID, StartTime, EndTime, Status	**S(BOOKING_FORM)** *//Phát khóa đọc trên BOOKING_FORM và trên điều kiện WHERE*

		FROM BOOKING_FORM WHERE CourtID = @CourtID AND StartTime >= @d AND StartTime < DATEADD(day,1,@d) AND StartTime < @EndTime AND EndTime > @StartTime;	
	.	**WAITFOR DELAY '00:00:10'**	
BEGIN TRAN			
Kiểm tra ngày, giờ, sân đó đã đặt chưa? IF EXISTS (SELECT 1 FROM BOOKING_FORM WHERE CourtID = @CourtID AND StartTime < @EndTime AND EndTime > @StartTime AND Status IN ('Confirmed','Pending')) BEGIN RAISERROR (N'Sân đã bị đặt	**S(BOOKING_FORM)** *//Phát khóa đọc trên BOOKING_FROM*		

trong khung giờ này', 16, 1); **ROLLBACK TRANSACTION;** RETURN; END			
Thêm đơn đặt sân DECLARE @NewBookingID NVARCHAR(8); SELECT @NewBookingID = 'BKG' + RIGHT('00000' + CAST(COUNT(*) + 1 AS VARCHAR(5)), 5) FROM BOOKING_FORM; INSERT INTO BOOKING_FORM (BookingID, CourtID, OrderDate, OrderTime, StartTime, EndTime, Status, CustomerID, SportCenterID, RID) VALUES (@NewBookingID, @CourtID, '2025-07-01', '15:00:00', @StartTime, @EndTime,	*Do T2 đã phát khóa S trên điều kiện WHERE nên nếu thêm dữ liệu mà nằm trong điều kiện where thì sẽ không phát khóa ghi được phải đợi T2 commit T1 mới được thêm đơn đặt sân*		

'Pending', 'CUS00001', 'CEN001', NULL);			
		Xem lại danh sách các sân đã đặt SELECT BookingID, CourtID, StartTime, EndTime, Status FROM BOOKING_FORM WHERE CourtID = @CourtID AND StartTime >= @d AND StartTime < DATEADD(day,1,@d) AND StartTime < @EndTime AND EndTime > @StartTime;	**S(BOOKING_FORM)** *//Phát khóa đọc trên BOOKINGFORM*
		COMMIT TRANSACTION	
Thêm đơn đặt sân DECLARE @NewBookingID NVARCHAR(8); SELECT @NewBookingID = 'BKG' + RIGHT('00000' + CAST(COUNT(*) + 1 AS VARCHAR(5)), 5) FROM BOOKING_FORM; INSERT INTO BOOKING_FORM (BookingID, CourtID, OrderDate,	**X(BOOKING_FORM)** *Do T2 đã COMMIT nên có thể phát khóa ghi*		

OrderTime, StartTime, EndTime, Status, CustomerID, SportCenterID, RID) VALUES (@NewBookingID, @CourtID, '2025-07-01', '15:00:00', @StartTime, @EndTime, 'Pending', 'CUS00001', 'CEN001', NULL);			
COMMIT TRANSACTION			

2. **Tình huống:** Nhân viên kỹ thuật A tạo đơn nghỉ phép mới. Nhân viên Quản lý B kiểm tra các đơn nghỉ phép đang chờ duyệt để duyệt.

ERR02: ***Phantom Read***

T1 (User = Nhân viên kỹ thuật): Thực hiện tạo một đơn nghỉ phép mới

T2 (User = Nhân viên Quản lý): Thực hiện kiểm tra sân các đơn nghỉ phép chưa duyệt

sp_TaoDonNghiPhep	**Khóa**	**sp_KiemTraCacDonNghiPhep**	**Khóa**
Input: @EmployeeID @WorkDate @WorkTime @Reason ***Output***: (1 - Thêm thành công, 0 - Thêm thất bại)		***Input***: @CenterID @CreateDate @Status ***Output***:: Các bản ghi trong **LEAVEFORM** thỏa điều kiện input	

		SET TRANSACTION ISOLATION LEVEL SERIALIZABLE	
		BEGIN TRAN	
	.	**Xem danh sách các đơn nghỉ phép** DECLARE @d DATE = CAST(@CreateDate AS DATE); SELECT lf.LeaveFormID, lf.EmployeeID, e.FullName, lf.WorkDate, lf.WorkTime, lf.DateCreate, lf.Reason, lf.Status FROM LEAVEFORM lf INNER JOIN EMPLOYEE e ON e.EmployeeID = lf.EmployeeID WHERE e.CenterID = @CenterID AND lf.DateCreate >= @d AND lf.DateCreate < DATEADD(DAY, 1, @d) AND (@Status IS NULL OR lf.Status = @Status) ORDER BY lf.DateCreate DESC, lf.LeaveFormID DESC;	**S(LEAVEFORM)** *//Phát khóa đọc trên LEAVFORM trên điều kiện WHERE*
	.	**WAITFOR DELAY '00:00:10'**	
BEGIN TRAN			
Kiểm tra các ràng buộc IF NOT EXISTS (SELECT 1	**S(LEAVEFORM)** *//Phát khóa đọc trên LEAVFORM*		

```
 FROM WORK_SCHEDULE ws

 WHERE

  ws.EmployeeID =
    @EmployeeID AND
    ws.WorkDate   = @WorkDate
    AND ws.WorkTime   =
    @WorkTime)

 BEGIN

        SELECT 0 AS Result,
N'Không tồn tại ca làm này trong
WorkSchedule.' AS Message;

        ROLLBACK;

        RETURN;

  END

IF EXISTS (

  SELECT 1

   FROM LEAVEFORM

   WHERE EmployeeID =
@EmployeeID AND WorkDate=
@WorkDate AND WorkTime=
@WorkTime AND Status=
N'Pending')

  BEGIN

        SELECT 0 AS Result,
N'Ca này đã có đơn Pending rồi.'
AS Message;

     ROLLBACK;

      RETURN;

 END
```

Thêm đơn nghỉ phép DECLARE @Next INT = (SELECT COUNT(*) + 1 FROM LEAVEFORM); DECLARE @LeaveFormID NVARCHAR(50) = N'LFM' + RIGHT('0000' + CAST(@Next AS VARCHAR(10)), 4); INSERT INTO LEAVEFORM (LeaveFormID, EmployeeID, WorkDate, WorkTime, DateCreate, Reason, Status) VALUES (@LeaveFormID, @EmployeeID, @WorkDate, @WorkTime, GETDATE(), @Reason, N'Pending'); SELECT 1 AS Result, N'Tạo đơn nghỉ thành công: ' + @LeaveFormID AS Message;	*Do T2 đang phát khóa đọc trên điều kiện where nên nếu thêm đơn nghỉ phép mà thỏa điều kiện WHERE thì sẽ không phát được khóa ghi phải đợi T2 commit thì mới thêm đơn nghỉ phép được.*		

		Xem lại danh sách các đơn nghỉ phép DECLARE @d DATE = CAST(@CreateDate AS DATE); SELECT lf.LeaveFormID, lf.EmployeeID, e.FullName, lf.WorkDate, lf.WorkTime, lf.DateCreate, lf.Reason, lf.Status FROM LEAVEFORM lf INNER JOIN EMPLOYEE e ON e.EmployeeID = lf.EmployeeID WHERE e.CenterID = @CenterID AND lf.DateCreate >= @d AND lf.DateCreate < DATEADD(DAY, 1, @d) AND (@Status IS NULL OR lf.Status = @Status) ORDER BY lf.DateCreate DESC, lf.LeaveFormID DESC;	**S(LEAVEFORM)** *//Phát khóa đọc trên LEAVFORM*
		COMMIT TRANSACTION	
Thêm đơn nghỉ phép DECLARE @Next INT = (SELECT COUNT(*) + 1 FROM LEAVEFORM);	**X(BOOKING_FORM)** *Do T2 đã commit nên phát khóa ghi bình thường*		

DECLARE @LeaveFormID NVARCHAR(50) = N'LFM' + RIGHT('0000' + CAST(@Next AS VARCHAR(10)), 4); INSERT INTO LEAVEFORM (LeaveFormID, EmployeeID, WorkDate, WorkTime, DateCreate, Reason, Status) VALUES (@LeaveFormID, @EmployeeID, @WorkDate, @WorkTime, GETDATE(), @Reason, N'Pending'); SELECT 1 AS Result, N'Tạo đơn nghỉ thành công: ' + @LeaveFormID AS Message;			
COMMIT TRANSACTION			

II. Sinh viên thực hiện: 23120087 - Phạm Ngọc Thanh

1. **Tình huống:** Khách hàng A đặt sân online và dùng dịch vụ S1 với số lượng đặt là Q1. Cùng lúc đó, Khách hàng B đặt sân online và dùng dịch vụ S1 với số lượng đặt là Q2.

ERR03: ***Lost update***

T1 (User = Khách hàng A): Đặt sân online và dùng dịch vụ S1 với số lượng đặt là Q1.

T2 (User = Khách hàng B): Đặt sân online và dùng dịch vụ S1 với số lượng đặt là Q2.

sp_DatSanTrucTuyen_1	Khóa	sp_DatSanTrucTuyen_2	Khóa
Input: @CourtID @CustomerID @SportCenterID @StartTime @EndTime @ServiceID @Quantity ***Output***: 1 - Thành công, 0 - Thất bại		***Input***: @CourtID @CustomerID @SportCenterID @StartTime @EndTime @ServiceID @Quantity ***Output***: 1 - Thành công, 0 - Thất bại	
SET TRANSACTION ISOLATION LEVEL READ COMMITTED		**SET TRANSACTION ISOLATION LEVEL READ COMMITTED**	
BEGIN TRAN			

Giả sử các điều kiện kiểm tra đã thoả mãn *//Tạo booking* INSERT INTO BOOKING_FORM (BookingID, CourtID, OrderDate, OrderTime, StartTime, EndTime, Status, CustomerID, SportCenterID) VALUES (@BookingID, @CourtID, CAST(GETDATE() AS DATE), CAST(GETDATE() AS TIME), @StartTime, @EndTime, N'Confirmed', @CustomerID, @SportCenterID);	**X(BOOKING_FORM)** *//Phát khoá ghi trên bảng BOOKING_FORM*		
//Tạo Service nếu có IF (@ServiceID IS NOT NULL AND @Quantity IS NOT NULL) BEGIN *//Cập nhật số lượng của Service* DECLARE @CurrentQty INT; //ĐỌC Quantity RA BIẾN @CurrentQty SELECT @CurrentQty =	**U(RENTING_EQUIPMENT)** *// Phát khoá Update Lock (U) trên bảng RENTING_EQUIPMENT*		

Quantity FROM RENTING_EQUIPMENT **WITH (UPDLOCK)** WHERE REServiceID = @ServiceID; IF @CurrentQty < @Quantity BEGIN RAISERROR (N'Số lượng dịch vụ không đủ', 16, 1); RETURN; END			
//Tạo mã SBID tăng dần… Thêm vào bảng SERVICE_BOOKING INSERT INTO SERVICE_BOOKING (SBID, BookingID, ServiceID, BookingQuantity) VALUES (@SBID, @BookingID, @ServiceID, @Quantity)	**X(SERVICE_BOOKING)** *//Phát khoá ghi trên bảng SERVICE_BOOKING*		
WAITFOR DELAY '00:00:10';			
		BEGIN TRAN	

		Giả sử các điều kiện kiểm tra đã thoả mãn *//Tạo booking* INSERT INTO BOOKING_FORM (BookingID, CourtID, OrderDate, OrderTime, StartTime, EndTime, Status, CustomerID, SportCenterID) VALUES (@BookingID, @CourtID, CAST(GETDATE() AS DATE), CAST(GETDATE() AS TIME), @StartTime, @EndTime, N'Confirmed', @CustomerID, @SportCenterID);	**X(BOOKING_FORM)** *//Phát khoá ghi trên bảng BOOKING_FORM*
		//Tạo Service_Booking nếu có IF (@ServiceID IS NOT NULL AND @Quantity IS NOT NULL) BEGIN *//Cập nhật số lượng của Service* DECLARE @CurrentQty INT; //ĐỌC Quantity RA BIẾN @CurrentQty SELECT @CurrentQty = Quantity FROM	**S(RENTING_EQUIPMENT)** *// Phát khoá đọc trên bảng RENTING_EQUIPMENT nhưng do sp_DatSanTrucTuyen_1 đang giữ UPDATE LOCK (UPDLOCK) nên không thể xin được, phải chờ COMMIT.*

		RENTING_EQUIPMENT WHERE REServiceID = @ServiceID; IF @CurrentQty < @Quantity BEGIN RAISERROR (N'Số lượng dịch vụ không đủ', 16, 1); RETURN; END	
// Ghi lại Quantity UPDATE RENTING_EQUIPMENT SET Quantity = @CurrentQty - @Quantity WHERE REServiceID = @ServiceID; END PRINT N'Đặt sân thành công';	**X(RENTING_EQUIPMENT)** *//Update khoá ghi trên bảng RENTING_EQUIPMENT*		
COMMIT TRANSACTION			
		//Tạo Service_Booking nếu có IF (@ServiceID IS NOT NULL AND @Quantity IS NOT NULL) BEGIN *//Cập nhật số lượng của Service*	**S(RENTING_EQUIPMENT)** *// Phát khoá đọc trên bảng RENTING_EQUIPMENT do sp_DatSanTrucTuyen_1 đã nhả khoá nên có thể xin được*

		DECLARE @CurrentQty INT; //ĐỌC Quantity RA BIẾN @CurrentQty SELECT @CurrentQty = Quantity FROM RENTING_EQUIPMENT WHERE REServiceID = @ServiceID; IF @CurrentQty < @Quantity BEGIN RAISERROR (N'Số lượng dịch vụ không đủ', 16, 1); RETURN; END	
		//Tạo mã SBID tăng dần… Thêm vào bảng SERVICE_BOOKING INSERT INTO SERVICE_BOOKING (SBID, BookingID, ServiceID, BookingQuantity) VALUES (@SBID, @BookingID, @ServiceID, @Quantity)	**X(SERVICE_BOOKING)** *//Phát khoá ghi trên bảng SERVICE_BOOKING*
		// Ghi lại Quantity	**X(RENTING_EQUIPMENT)**

		UPDATE RENTING_EQUIPMENT SET Quantity = @CurrentQty - @Quantity WHERE REServiceID = @ServiceID; END PRINT N'Đặt sân thành công';	*//Update khoá ghi trên bảng RENTING_EQUIPMENT*
		COMMIT TRANSACTION	

2. **Tình huống:** Quản lý A nâng số tiền thuê sân C1 thêm 5%. Cùng lúc đó, quản lý B giảm số tiền thuê sân C1 xuống 5%.

ERR04: ***Lost update***

T1 (User = Quản lý A): Nâng số tiền thuê sân C1 thêm 5%.

T1 (User = Quản lý B): Giảm số tiền thuê sân C1 xuống 5%.

sp_UpdateUnitPrice_1	**Khóa**	**sp_UpdateUnitPrice_2**	**Khóa**
Input: @CourtID @UnitPrice @Percentage1 = 1.05 ***Output***: 1 - Thành công, 0 - Thất bại		***Input***: @CourtID @UnitPrice @Percentage2 = 0.95 ***Output***: 1 - Thành công, 0 - Thất bại	
SET TRANSACTION ISOLATION LEVEL READ COMMITTED		**SET TRANSACTION ISOLATION LEVEL READ COMMITTED**	
BEGIN TRAN			

Kiểm tra mã sân có tồn tại hay không IF NOT EXISTS (SELECT 1 FROM SPORT_COURT **WITH (UPDLOCK)** WHERE SPORT_COURT.CourtID = @CourtID) BEGIN PRINT N'DO NOT EXIST COURT.' **ROLLBACK TRANSACTION;** RETURN END	**U(SPORT_COURT)** *//Phát khoá UPDATE LOCK (UPDLOCK) trên bảng SPORT_COURT với đk CourtID = @CourtID*		
Update giá cơ bản lên 5% SELECT @UnitPrice = UnitPrice FROM SPORT_COURT WHERE SPORT_COURT.CourtID = @CourtID SET @Price = @Price * @Percentage1;	**S(SPORT_COURT)**		
WAITFOR DELAY '00:00:10'			
		BEGIN TRAN	
		Kiểm tra mã sân có tồn tại hay không IF NOT EXISTS	**S(SPORT_COURT)** *//Phát khoá đọc*

		(SELECT 1 FROM SPORT_COURT WHERE SPORT_COURT.CourtID = @CourtID) BEGIN PRINT N'DO NOT EXIST COURT.' **ROLLBACK TRANSACTION;** RETURN END	*trên bảng SPORT_COURT với đk CourtID = @CourtID* ***nhưng do sp_UpdateUnitPrice_1 đang giữ khoá UPDATE LOCK (UPDLOCK) nên không thể xin được***
Ghi xuống database UPDATE SPORT_COURT SET UnitPrice = @UnitPrice WHERE CourtID = @CourtID	**X(SPORT_COURT)** *//Phát khoá ghi trên bảng SPORT_COURT với đk CourtID = @CourtID*		
COMMIT TRANSACTION			
		Kiểm tra mã sân có tồn tại hay không IF NOT EXISTS (SELECT 1 FROM SPORT_COURT WHERE SPORT_COURT.CourtID = @CourtID) BEGIN PRINT N'DO NOT EXIST COURT.' **ROLLBACK TRANSACTION;** RETURN END	**S(SPORT_COURT)** *//Phát khoá đọc trên bảng SPORT_COURT với đk CourtID = @CourtID* ***do sp_UpdateUnitPrice_1 đã nhả khoá nên có thể xin được***

		Update giá cơ bản lên 5% SELECT @UnitPrice = UnitPrice FROM SPORT_COURT WHERE SPORT_COURT.CourtID = @CourtID SET @Price = @Price * @Percentage2;	**S(SPORT_COURT)**
		Ghi xuống database UPDATE SPORT_COURT SET UnitPrice = @UnitPrice WHERE CourtID = @CourtID	**X(SPORT_COURT)** *//Phát khoá ghi trên bảng SPORT_COURT với đk CourtID = @CourtID*
		COMMIT TRANSACTION	

III. Sinh viên thực hiện: 23120361 - Đào Đức Thịnh

1. **Tình huống:** Khách online A tạo phiếu đặt sân B, chưa commit. Nhân viên lễ tân kiểm tra tình trạng sân B.

ERR05: ***Dirty read***

T1 (User = Khách hàng online): Thực hiện tạo phiếu đặt sân B, chưa commit

T2 (User = Nhân viên lễ tân): Thực hiện kiểm tra tình trạng sân B

sp_DatSanTrucTuyen	**Khóa**	**SP_GetAllBookings**	**Khóa**
Input: @CourtID @CustomerID @SportCenterID @StartTime @EndTime @ServiceName = NULL @Quantity = NULL ***Output***: thông báo thêm thành công, thông báo thêm thất bại		***Input***: ***Output***:: Tất cả các bản ghi trong **BOOKING_FORM**	
		SET TRANSACTION ISOLATION LEVEL READ COMMITTED	
BEGIN TRAN			
Kiểm tra ngày, giờ IF (@StartTime >= @EndTime) BEGIN RAISERROR (N'Thời gian đặt sân không hợp lệ', 16, 1);	**S(BOOKING_FORM)** *//Phát khóa đọc trên BOOKING_FROM.*		

ROLLBACK TRANSACTION; RETURN; END			
Thêm đơn đặt sân INSERT INTO BOOKING_FORM (BookingID, CourtID, OrderDate, OrderTime, StartTime, EndTime,Status, CustomerID, SportCenterID) VALUES (@BookingID, @CourtID, CAST(GETDATE() AS DATE), CAST(GETDATE() AS TIME),@StartTime, @EndTime, N'Confirmed', @CustomerID, @SportCenterID);	**X(BOOKING_FORM)** *//Nâng cấp thành khóa ghi để thêm dữ liệu*		
WAITFOR DELAY '00:00:15'			
ROLLBACK			
		BEGIN TRAN	
		Xem danh sách các sân đã đặt SELECT BF.BookingID, BF.CourtID, SC.CourtType, CT.CourtTypeName, BF.OrderDate,	**S(BOOKING_FORM)** *//Phát khóa đọc trên BOOKINGFORM.* *// Đợi T1 kết thúc ghi rồi phát khóa đọc trên BOOKINGFORM*

		BF.OrderTime, BF.StartTime, BF.EndTime, BF.Status, C.FullName AS CustomerName, C.PhoneNumber, C.CustomerID FROM BOOKING_FORM BF INNER JOIN SPORT_COURT SC ON BF.CourtID = SC.CourtID INNER JOIN COURT_TYPE CT ON SC.CourtType = CT.CourtType LEFT JOIN CUSTOMER C ON BF.CustomerID = C.CustomerID WHERE CAST(BF.OrderDate AS DATE) >= CAST(GETDATE() AS DATE) ORDER BY BF.OrderDate, BF.StartTime;	
		COMMIT	

2. **Tình huống:** Quản lý cập nhật tăng đơn giá sân B từ 300k lên 500k tại T1 nhưng chưa commit. Tại T2, nhân viên truy vấn giá sân B, hệ thống trả về 500k. Nhân viên dùng giá này tính tiền cho khách. Tại T1, Quản lý thực hiện Rollback.

ERR06: ***Dirty read***

T1 (User = Quản lý): Thực hiện cập nhật đơn giá sân, chưa commit và sau đó Rollback.

T2 (User = Nhân viên lễ tân): Thực hiện kiểm tra giá sân để báo giá/lập hóa đơn cho khách.

sp_CapNhatGiaSan	**Khóa**	**sp_XemGiaSan**	**Khóa**
Input: @CourtID @NewPrice ***Output:*** (1 - Thành công, 0 - Thất bại)		***Input:*** @CourtID ***Output:*** Đơn giá của sân	
		SET TRANSACTION ISOLATION LEVEL READ COMMITTED;	
BEGIN TRAN			
Cập nhật giá sân mới UPDATE SPORT_COURT SET UnitPrice = @NewPrice WHERE CourtID = @CourtID;	**X(SPORT_COURT)** *//Phát khóa ghi trên dòng dữ liệu.*		
WAITFOR DELAY '00:00:10'			
		BEGIN TRAN	
		Xem đơn giá sân SELECT UnitPrice	*Đợi T1 kết thúc* *//T2 bị chặn do không được đọc dữ*

		FROM SPORT_COURT WHERE CourtID = @CourtID;	*liệu chưa commit.*
ROLLBACK	*//Giải phóng WL*		
		Xem đơn giá sân SELECT UnitPrice FROM SPORT_COURT WHERE CourtID = @CourtID;	**S(SPORT_COURT)** //Phát khóa đọc và lấy được giá trị đã nhất quán.
		COMMIT	

IV. Sinh viên thực hiện: 23120411 - Thạch Ý

1. **Tình huống:** Lễ tân thực hiện tạo Booking Form. Nhân viên kỹ thuật cập nhật trạng thái sân

ERR07: Unrepeatable Read

T1 (User = Nhân viên Lễ tân): Tạo Booking Form

T2 (User = Nhân viên Kỹ thuật): Cập nhật Trạng thái Sân

sp_Receptionist_Booking_Create	**Khóa**	**sp_TN1_Tech_UpdateCourtStatusOnly**	**Khóa**
Input: @CourtID @CustomerID @SportCenterID @RID @StartDate @StartTime_In @EndTime_In @ServiceID @Quantity ***Output:*** (1 (Thành công), -1, -2, -3, -4, -5, -6, -7, -8, -9, -10, -11, -12)		***Input:*** @CenterID @CourtID @Status ***Output:*** (1 - Thành công , 0 - Thất bại)	
SET TRANSACTION ISOLATION **LEVEL REPEATABLE READ**		**SET TRANSACTION ISOLATION** **LEVEL READ UNCOMMITTED;**	
BEGIN TRAN			

Kiểm tra xung đột lịch đặt sân IF EXISTS (SELECT 1 FROM BOOKING_FORM WHERE CourtID = @CourtID AND Status <> N'Cancelled' AND ((@FullStartTime >= StartTime AND @FullStartTime < EndTime) OR (@FullEndTime > StartTime AND @FullEndTime <= EndTime) OR (StartTime >= @FullStartTime AND EndTime <= @FullEndTime))) BEGIN RAISERROR(N'Lỗi: Sân này đã được đặt trong khoảng thời gian bạn chọn!', 16, 1); RETURN -10; END	**S(BOOKING_FORM)** *//Thiết lập S-Lock (Khóa đọc) trên BOOKING_FORM và Giữ khóa đọc (S-Lock) cho tới khi giao tác T1 kết thúc (COMMIT)*		
Kiểm tra trạng thái sân lần 1 SELECT @StatusCheck1 = Status FROM SPORT_COURT WHERE CourtID = @CourtID;	**S(SPORT_COURT)** *//Thiết lập S-Lock (Khóa đọc) trên SPORT_COURT và Giữ khóa đọc (S-Lock) cho tới khi giao tác T1 kết thúc (COMMIT)*		
WAITFOR DELAY '00:00:10'			

		BEGIN TRAN	
		Cập nhật trạng thái sân UPDATE dbo.SPORT_COURT SET Status = LTRIM(RTRIM(@Status)) WHERE CourtID = @CourtID AND CenterID = @CenterID; **CHỜ**	*T2 xen vào và cố gắng UPDATE sân, T2 cần X-Lock. Nhưng vì T1 đang giữ S-Lock (và S-Lock xung đột với X-Lock), T2 bị chặn và phải chờ.*
Kiểm tra trạng thái sân lần 2 SELECT @StatusCheck2 = Status FROM SPORT_COURT WHERE CourtID = @CourtID; IF (@StatusCheck1 <> @StatusCheck2) BEGIN RAISERROR(N'LỖI: Trạng thái sân đã bị thay đổi trong khi đang xử lý! (Unrepeatable Read)', 16, 1); **ROLLBACK TRAN;** RETURN -12; END	**S(SPORT_COURT)** *T1 không cần xin khóa vì khóa S-Lock hiện tại vẫn đang được T1 giữ. Nếu **COMMIT** thì khóa được nhả* .		

Tạo Booking_form INSERT INTO BOOKING_FORM (BookingID, CourtID, OrderDate, OrderTime, StartTime, EndTime, Status, CustomerID, SportCenterID, RID) VALUES (@NewBKG_ID, @CourtID, GETDATE(), CAST(GETDATE() AS TIME), @FullStartTime, @FullEndTime, N'Pending', @CustomerID, @SportCenterID, @RID);	**X(BOOKING_FORM)** //Phát khóa ghi trên BOOKING_FORM		
Tạo Service_Booking INSERT INTO SERVICE_BOOKING (SBID, BookingID, ServiceID, BookingQuantity) VALUES (@NewSBID, @NewBKG_ID, @ServiceID, @Quantity);	**X(SERVICE_BOOKING)** //Phát khóa ghi trên SERVICE_BOOKING		
COMMIT TRAN;			
		Cập nhật trạng thái sân UPDATE dbo.SPORT_COURT SET Status = LTRIM(RTRIM(@Status)) WHERE CourtID = @CourtID AND CenterID = @CenterID;	**X(SPORT_COURT)** *//T2 phát khóa X trên hàng SPORT_COURT và giữ đến COMMIT.*

		COMMIT	

2. **Tình huống :** Admin A cập nhật tài khoản nhân viên, Admin B xóa tài khoản nhân viên

ERR08: Unrepeatable Read

T1 (Admin 1): Cập nhật tài khoản nhân viên

T2 (Admin 2): Xóa tài khoản nhân viên

sp_Admin_AccountEmployees_UpdateAccount	**Khóa**	**sp_Admin_AccountEmployees_DeleteAccount**	**Khóa**
Input: @EmployeeID @NewUsername @NewPassword ***Output:*** (1 - Thành công, -1, -2, -3, -4)		***Input:*** @EmployeeID ***Output:*** (1 - Thành công ,-1, -2)	
SET TRANSACTION ISOLATION **LEVEL REPEATABLE READ;**			
BEGIN TRAN			
Kiểm tra tài khoản nhân viên lần 1 DECLARE @OldUsername NVARCHAR(50) = (SELECT AccountLogin FROM EMPLOYEE WHERE	**S(EMPLOYEE)** *//Phát khóa đọc trên EMPLOYEE và Giữ khóa đọc (S-Lock) cho tới khi giao tác T1 kết thúc*		

EmployeeID = @EmployeeID); IF @OldUsername IS NULL BEGIN PRINT N'Lỗi: Nhân viên này chưa có tài khoản để cập nhật. Hãy dùng nút Create.'; RETURN -2; END	*(COMMIT)*		
WAITFOR DELAY '00:00:10'			
		BEGIN TRAN	
		Cập nhật tài khoản nhân viên là NULL UPDATE EMPLOYEE SET AccountLogin = NULL WHERE EmployeeID = @EmployeeID; **CHỜ**	*T2 xen vào và cố gắng UPDATE AccountLogin(EMPLOYEE) , T2 cần X-Lock. Nhưng vì T1 đang giữ S-Lock (và S-Lock xung đột với X-Lock), T2 bị chặn và phải chờ*
Kiểm tra tài khoản nhân viên lần 2 DECLARE @OldUsername1 NVARCHAR(50) = (SELECT AccountLogin FROM EMPLOYEE WHERE EmployeeID = @EmployeeID); IF @OldUsername1 IS NULL BEGIN	**S(EMPLOYEE)** *T1 không cần xin khóa vì khóa S-Lock hiện tại vẫn đang được T1 giữ. Nếu COMMIT thì khóa được nhả*		

PRINT N'Lỗi:(Unrepeatable Read) Nhân viên này chưa có tài khoản để cập nhật. Hãy dùng nút Create.'; **ROLLBACK TRAN;** RETURN -4; END			
Cập nhật mật khẩu UPDATE ACCOUNT_LOGIN SET Password = @NewPassword WHERE AccountLogin = @OldUsername;	**X(ACCOUT_LOGIN)** //Phát khóa ghi trên ACCOUNT_LOGIN		
COMMIT TRANSACTION;			
		Cập nhật tài khoản nhân viên là NULL UPDATE EMPLOYEE SET AccountLogin = NULL WHERE EmployeeID = @EmployeeID;	**X(EMPLOYEE)** *//T2 phát khóa X trên hàng EMPLOYEE và giữ đến COMMIT.*
		COMMIT TRANSACTION;	

V. Sinh viên thực hiện: 23120028 - Nguyễn Hoàng Danh

1. **Tình huống:** Lễ tân chỉnh sửa sân khi khách hàng đang hủy

ERR09: ***Deadlock***

T1 (User = Quản lý): Phân công lịch làm việc

T2 (User = Nhân viên kỹ thuật): Tạo đơn xin nghỉ phép

sp_QL3_Manager_AssignWorkSchedule	**Khóa**	**sp_TN4_Tech_AddLeaveForm**	**Khóa**
Input: @CenterID @EmployeeID @WorkDate @WorkTime @Shift @MID @result		***Input***: @EmployeeID @WorkDate @WorkTime @Reason	
SET TRANSACTION ISOLATION LEVEL REPEATEABLE READ;			
BEGIN TRAN			
Kiểm tra nhân viên đã được chấp nhận cho nghỉ ca này IF EXISTS (SELECT 1 FROM LEAVEFORM lf	**S(LEAVEFORM)** *//phát khóa đọc khi đọc dữ liệu*		

WHERE lf.EmployeeID = @EmployeeID AND lf.Status = 'Approved' AND lf.WorkDate = @WorkDate AND lf.WorkTime = @WorkTime) BEGIN SET @result = -7; RETURN; END			
WAITFOR DELAY '00:00:10'			
		Cài đặt vòng lặp while để chạy lại giao tác nếu gặp deadlock: DECLARE @retry_count INT = 0; DECLARE @MaxRetries INT = 3; DECLARE @Success BIT = 0; -- Biến cờ kiểm soát thành công WHILE @retry_count < @MaxRetries AND @Success = 0	
		BEGIN TRAN	
		Kiểm tra tồn tại ca làm IF NOT EXISTS (SELECT 1	**S(WORK_SCHEDULE)** *//Phát khóa ghi trên* WORK_SCHEDU

		FROM WORK_SCHEDULE ws WITH (TABLOCK,HOLDLOCK) WHERE ws.EmployeeID = @EmployeeID AND ws.WorkDate = @WorkDate AND ws.WorkTime = @WorkTime) BEGIN SELECT 0 AS Result, N'Không tồn tại ca làm này trong WorkSchedule.' AS Message; ROLLBACK; RETURN; END	LE
		Tạo đơn xin nghỉ phép DECLARE @Next INT = (SELECT COUNT(*) + 1 FROM LEAVEFORM); DECLARE @LeaveFormID NVARCHAR(50) = N'LFM' + RIGHT('0000' + CAST(@Next AS VARCHAR(10)), 4); INSERT INTO LEAVEFORM (LeaveFormID, EmployeeID, WorkDate, WorkTime, DateCreate, Reason, Status) VALUES (@LeaveFormID,	**X(LEAVEFORM)** *//Phát khóa ghi trên LEAVEFORM KHÔNG THÀNH CÔNG do đã tồn tại khóa ghi của T1 trên LEAVEFORM* **=> WAIT**

		@EmployeeID, @WorkDate, @WorkTime, GETDATE(), @Reason, N'Pending');	
Thêm ca làm việc INSERT INTO WORK_SCHEDULE (EmployeeID, WorkDate, WorkTime, Shift, MID) VALUES (@EmployeeID, @WorkDate, @WorkTime, @Shift, @MID);	**X(WORK_SCHEDULE)** *//Phát khóa ghi trên WORK_SCHEDULE kHÔNG THÀNH CÔNG do đã tồn tại khóa ghi của T2 trên WORK_SCHEDULE* **=> WAIT**		
		ROLLBACK	
Tiếp tục thêm ca làm việc INSERT INTO WORK_SCHEDULE (EmployeeID, WorkDate, WorkTime, Shift, MID) VALUES (@EmployeeID, @WorkDate, @WorkTime, @Shift, @MID);	**X(WORK_SCHEDULE)** *//Phát khóa ghi trên WORK_SCHEDULE*		
COMMIT			
		Khởi động lại T2 sau 1 khoảng thời gian nhờ vòng lặp while: WAITFOR DELAY '00:00:00.500'	
		BEGIN TRAN	
		Kiểm tra tồn tại ca làm	**S(WORK_SCHEDULE)**

		IF NOT EXISTS (SELECT 1 FROM WORK_SCHEDULE ws WITH (TABLOCK,HOLDLOCK) WHERE ws.EmployeeID = @EmployeeID AND ws.WorkDate = @WorkDate AND ws.WorkTime = @WorkTime) BEGIN SELECT 0 AS Result, N'Không tồn tại ca làm này trong WorkSchedule.' AS Message; ROLLBACK; RETURN; END	*//Phát khóa ghi trên* WORK_SCHEDULE
		Tạo đơn xin nghỉ phép DECLARE @Next INT = (SELECT COUNT(*) + 1 FROM LEAVEFORM); DECLARE @LeaveFormID NVARCHAR(50) = N'LFM' + RIGHT('0000' + CAST(@Next AS VARCHAR(10)), 4); INSERT INTO LEAVEFORM (LeaveFormID, EmployeeID, WorkDate, WorkTime, DateCreate,	**X(LEAVEFORM)** *//Phát khóa ghi trên LEAVEFORM*

		Reason, Status) VALUES (@LeaveFormID, @EmployeeID, @WorkDate, @WorkTime, GETDATE(), @Reason, N'Pending');	
		COMMIT	

2. **Tình huống:** Lễ tân chỉnh sửa sân đặt khi khách hàng đang đổi giờ đặt sân

ERR10: ***Deadlock***

T1 (User = Khách hàng online): Đăng kí tài khoản

T2 (User = Khách hàng online): Đăng kí tài khoản

sp_DangKyKhachHang	**Khóa**	**sp_DangKyKhachHang**	**Khóa**
Input: @FullName @Birthday @PhoneNumber @Email @Address @AccountLogin @Password		***Input***: @FullName @Birthday @PhoneNumber @Email @Address @AccountLogin @Password	
SET TRANSACTION ISOLATION LEVEL REPEATEABLE READ			
BEGIN TRAN			

Kiểm tra account có tồn tại chưa IF EXISTS (SELECT 1 FROM ACCOUNT_LOGIN_CUSTOMER WITH(TABLOCK, HOLDLOCK) WHERE AccountLoginC = @AccountLogin) BEGIN RAISERROR (N'Tên đăng nhập đã tồn tại', 16, 1); RETURN; END	**S(ACCOUNT_LOGIN_CUSTOMER)** *//Phát khóa đọc trên* ACCOUNT_LOGIN_CUSTOMER		
WAITFOR DELAY '00:00:10'			
		Cài đặt vòng lặp while để chạy lại giao tác nếu gặp deadlock: DECLARE @retry_count INT = 0; DECLARE @MaxRetries INT = 3; DECLARE @Success BIT = 0; -- Biến cờ kiểm soát thành công WHILE @retry_count < @MaxRetries AND @Success = 0	
		BEGIN TRAN	
		Kiểm tra account có tồn tại chưa IF EXISTS (	**S(ACCOUNT_LOGIN_CUSTOMER)**

		SELECT 1 FROM ACCOUNT_LOGIN_CUSTOMER WITH(TABLOCK, HOLDLOCK) WHERE AccountLoginC = @AccountLogin) BEGIN RAISERROR (N'Tên đăng nhập đã tồn tại', 16, 1); RETURN; END	*//Phát khóa đọc trên* ACCOUNT_LOGIN_CUSTOMER
		Tạo tài khoản đăng nhập ACCOUNT_LOGIN_CUSTOMER(AccountLoginC, Password) VALUES (@AccountLogin, @Password);	**X(ACCOUNT_LOGIN_CUSTOMER)** //Phát khóa ghi trên ACCOUNT_LOGIN_CUSTOMER KHÔNG THÀNH CÔNG do đã có khóa đọc của T1 =>Wait
Tạo tài khoản đăng nhập ACCOUNT_LOGIN_CUSTOMER(AccountLoginC, Password) VALUES (@AccountLogin, @Password);	**X(ACCOUNT_LOGIN_CUSTOMER)** //Phát khóa ghi trên ACCOUNT_LOGIN_CUSTOMER KHÔNG THÀNH CÔNG do đã có khóa đọc của T2		

	=>Wait		
		ROLLBACK	
Tiếp tục tạo tài khoản đăng nhập ACCOUNT_LOGIN_CUSTOMER(AccountLoginC, Password) VALUES (@AccountLogin, @Password);	**X(ACCOUNT_LOGIN_CUSTOMER)** //Phát khóa ghi trên ACCOUNT_LOGIN_CUSTOMER		
COMMIT			
		Khởi động lại T2 sau 1 khoảng thời gian nhờ vòng lặp while: WAITFOR DELAY '00:00:00.500'	
		BEGIN TRAN	
		Kiểm tra account có tồn tại chưa IF EXISTS (SELECT 1 FROM ACCOUNT_LOGIN_CUSTOMER WITH(TABLOCK, HOLDLOCK) WHERE AccountLoginC = @AccountLogin) BEGIN RAISERROR (N'Tên đăng nhập đã tồn tại', 16, 1); RETURN; END	**S(ACCOUNT_LOGIN_CUSTOMER)** *//Phát khóa đọc trên* ACCOUNT_LOGIN_CUSTOMER
		Tạo tài khoản đăng nhập	**X(ACCOUNT_LOGIN_CUST**

		ACCOUNT_LOGIN_CUSTOMER(AccountLoginC, Password) VALUES (@AccountLogin, @Password);	**OMER)** //Phát khóa ghi trên ACCOUNT_LOGIN_CUSTOMER
		COMMIT	

- **Link:** **Source app và video demo**